

Multi-Camera ALPR Vehicle Trajectory Reconstruction System

Complete Technology Guide — every technology, library, API, database table and design decision, explained

Project: **alpr-university-gate** · Platform: NVIDIA Jetson Orin (JetPack R39, Ubuntu 24.04, aarch64)

Document date: 13 September 2026

Contents

1. What the system does
2. Hardware and platform
3. Architecture and data flow
4. Technology stack at a glance
5. The ALPR pipeline (computer vision and AI)
6. Multi-camera layer (uploads, RTSP, sequential processing)
7. Database and SQL
8. Trajectory reconstruction and GIS
9. Maps and the traffic heatmap
10. Vehicle profiles and body-colour detection
11. Vehicle search (natural-language parser)
12. Blacklisted vehicles
13. REST API (FastAPI)
14. Dashboards and live video
15. Configuration files
16. Project structure
17. Running the system: processes, ports, commands
18. Measured performance
19. Testing
20. Known limitations
21. Glossary

1. What the system does

The system reads vehicle number plates from video (recorded files or live RTSP cameras), records every detection in a database with the camera's location, and reconstructs where each vehicle travelled across a network of cameras.

Capability	What it means in practice
Automatic Licence Plate Recognition (ALPR)	Finds vehicles, finds their plates, reads the plate text with a neural OCR model, and stores the result.
Multi-camera processing	Four configured camera sites (India Gate, Connaught Place, Karol Bagh, Kashmere Gate). Each takes an uploaded video or a live RTSP stream. Uploaded videos are processed one after another; RTSP streams all run together and keep running until the session is stopped.
Camera management	Camera sites are added and removed from the dashboard: adding asks for the place, latitude and longitude, and the new camera immediately gets its own upload/RTSP controls, feed panel, map marker and queue position. Cameras can also be renamed in place. Removing takes its feed with it and keeps its recorded detections.
Trajectory reconstruction	For any plate, orders its sightings in time and draws the route on a map with distances, travel times and speeds.
Vehicle profiles	One summary per vehicle: YOLO vehicle class, body colour, thumbnails, first/last seen, cameras visited, visit history.
Vehicle search	Type a description (“white cars after 9am today at India Gate”) and see matching vehicles; click through to their trajectory.
Traffic heatmap	2-D map coloured by where vehicles are now (or by total traffic per camera).
Blacklist alerts	Plates on a watch list (e.g. DL7CD5017) raise a red alert on the dashboard and in the live processing log.
Live camera wall	Four feeds per row of each camera's annotated video, streamed smoothly to the browser; a fifth camera starts the next row.
REST API	Every capability is also available as HTTP/JSON endpoints for other software.

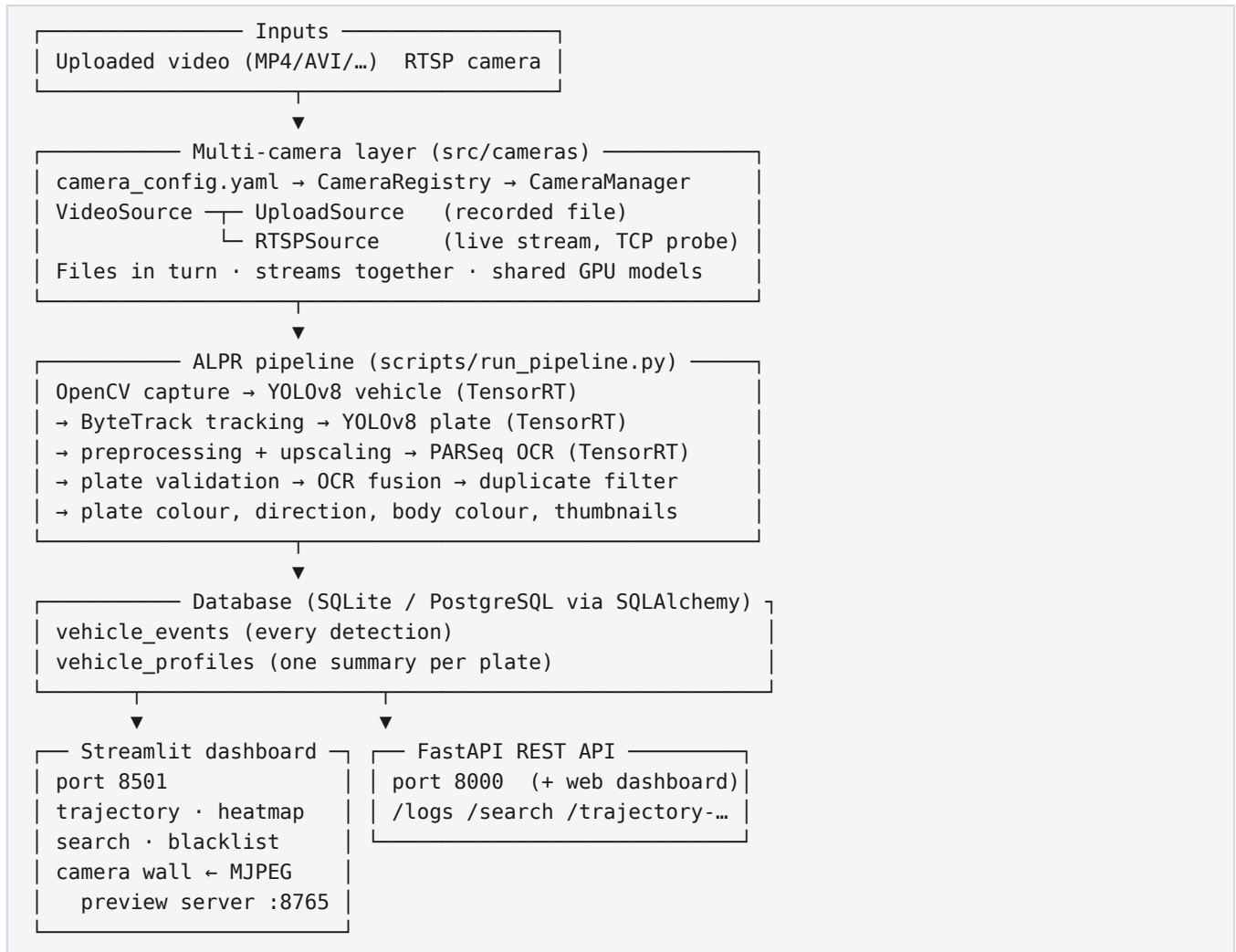
2. Hardware and platform

Component	Value on this device	Why it matters
Computer	NVIDIA Jetson Orin (“Orin (nvgpu)”)	An edge AI computer with an integrated NVIDIA GPU. All inference runs locally — no cloud, no internet required.
CPU architecture	aarch64 (64-bit ARM)	Python wheels must be ARM builds; some packages (TensorRT, PyCUDA) come from JetPack rather than pip.
JetPack / L4T	R39 (release), revision 2.0	NVIDIA's board support package: Linux kernel, GPU driver, CUDA, TensorRT.
Operating system	Ubuntu 24.04.4 LTS	Base OS supplied by JetPack.
CUDA	13.2 (V13.2.78)	NVIDIA's GPU computing platform used by TensorRT, PyTorch and PyCUDA.
Python	3.12.3, in virtual environment / home/ansh/alpr/alpr_env	All application code is Python.

Jetson detail: TensorRT is installed by JetPack into `/usr/lib/python3.12/dist-packages`, which a virtual environment cannot see by default. `src/utis/jetson_paths.py` adds that path at startup; without it Ultralytics tries to download TensorRT from the internet and hangs.

3. Architecture and data flow

3.1 Component architecture



3.2 Data flow for one vehicle

- | | |
|---|----------------------------------|
| 1. Frame read from video/RTSP | (OpenCV + FFmpeg) |
| 2. Vehicles detected in the frame | (YOLOv8n, TensorRT FP16) |
| 3. Each vehicle given a persistent track ID | (ByteTrack) |
| 4. Plate located inside the vehicle crop | (custom YOLOv8 plate model) |
| 5. Plate image cleaned and enlarged | (CLAHE, gamma, sharpen, upscale) |
| 6. Plate text read | (PARSeq-tiny, TensorRT) |
| 7. Text validated against Indian plate formats | (regex + OCR correction) |
| 8. 5 readings of the same track voted together | (OCR fusion) |
| 9. Duplicate check (same plate within 30 s) | (Levenshtein distance ≤ 2) |
| 10. Plate colour → registration category; direction; YOLO class; body colour | |
| 11. Images and thumbnails saved to disk | |
| 12. Row inserted into vehicle_events; vehicle profile updated in same transaction | |
| 13. Blacklist checked → alert in live log / dashboard | |
| 14. Dashboard and API read the database; trajectory rebuilt on demand | |

4. Technology stack at a glance

Category	Technology	Version	Used for
Language	Python	3.12.3	All application code
Object detection	Ultralytics YOLOv8	8.4.93	Vehicle detection (COCO-pretrained yolov8n) and plate detection (custom-trained model)
Inference engine	NVIDIA TensorRT	10.16.2.10	Runs the YOLO and PARSeq models as optimised GPU “engines” (FP16)
GPU bindings	PyCUDA	JetPack build	GPU memory buffers and CUDA streams for the PARSeq TensorRT engine
Deep learning	PyTorch	2.13.0 (CUDA 13.2)	Used by Ultralytics; model training
Tracking	Supervision (ByteTrack)	0.25.0	Keeps a stable ID per vehicle across frames
OCR model	PARSeq (tiny)	official baudm weights, fine-tuned	Reads plate characters
Fallback OCR	RapidOCR + ONNX Runtime	ORT 1.20.1	CPU OCR backend (configurable)
Image processing	OpenCV (cv2)	4.10.0	Video capture, RTSP, resizing, colour spaces, JPEG encoding, drawing
Numerics	NumPy	1.26.4	Image arrays, colour statistics
Database	SQLite (default) / PostgreSQL (supported)	SQLite 3	Stores detections and vehicle profiles
ORM / SQL toolkit	SQLAlchemy	2.0.30	Tables, queries, sessions, schema migration
Web API	FastAPI	0.115.0	REST endpoints
ASGI framework	Starlette	0.37.2	FastAPI's foundation; static file serving
ASGI server	Uvicorn	0.32.0	Runs the FastAPI app
Validation	Pydantic	2.9.2	Request/response schemas
Dashboard	Streamlit	1.39.0	Main operator dashboard
Data frames	Pandas	2.2.3	Tables in the original dashboard
Maps	Leaflet.js	1.9.4	Interactive trajectory map and heatmap
Heatmap	Leaflet.heat	0.2.0	Heat layer on the map
Map tiles	OpenStreetMap, Esri World Imagery	—	Street map and satellite layers
Live video to browser	MJPEG over HTTP (Python http.server)	standard library	Smooth camera wall video
Configuration	YAML (PyYAML)	6.0.2	config.yaml, camera_config.yaml, blacklist.yaml
Testing	pytest, Hypothesis, Streamlit AppTest, FastAPI TestClient	pytest 8.2.0	665 automated tests

5. The ALPR pipeline (computer vision and AI)

Implemented in `scripts/run_pipeline.py` with components under `src/`. Each stage is described with the technology it uses and why.

5.1 Frame capture — OpenCV + FFmpeg

`src/capture/frame_capture.py`. OpenCV's `VideoCapture` (backed by FFmpeg) opens local files, RTSP/RTMP/HTTP streams, or webcams.

- **Recorded files** are read frame by frame in order (`frame_skip: 1` — every frame).
- **Live streams** use a background reader thread that keeps only the newest frame, so latency cannot build up when processing is slower than the camera.
- Automatic reconnection (5 retries) and stall detection (10 s without a frame).

5.2 Vehicle detection — YOLOv8 + TensorRT

YOLO (“You Only Look Once”) is a single-pass convolutional neural network that outputs bounding boxes and class labels. The vehicle detector is **YOLOv8n** pretrained on the COCO dataset; only the classes *car*, *truck*, *bus*, *motorcycle* are kept.

Setting	Value
Model file	<code>models/vehicle_detector/yolov8n.engine</code>
Input size	480 px
Confidence threshold	0.5
Precision	FP16 (half precision)

TensorRT is NVIDIA's inference optimiser. A trained model (`.pt` / `.onnx`) is compiled once into an `.engine` file tuned for this exact GPU: layers are fused, kernels are chosen for the hardware, and weights run in 16-bit floating point. On Jetson this is several times faster than running the PyTorch model directly.

5.3 Vehicle tracking — ByteTrack

`src/tracking/vehicle_tracker.py` uses ByteTrack from the **Supervision** library. It links detections in consecutive frames into tracks, giving each physical vehicle a stable *track ID*. This is what lets several OCR readings of the same car be combined, and lets a vehicle be stored once per pass instead of once per frame. Lost tracks are kept for 60 frames; the IoU matching threshold is 0.5.

5.4 Plate detection — custom YOLOv8

`src/detection/plate_detector.py`. A YOLOv8 model trained on number-plate images runs *inside each vehicle crop* (not the whole frame), which makes it faster and more accurate. Engine `models/plate_detector/best.engine`, input 320 px, confidence 0.4.

5.5 Plate preprocessing and super-resolution

`src/preprocessing/plate_preprocessor.py` and `src/enhancement/super_resolution.py`:

- **CLAHE** (Contrast-Limited Adaptive Histogram Equalisation) — boosts local contrast so characters stand out.
- **Adaptive gamma** — brightens dark/night plates, tames glare.
- **Sharpening** — crisper character edges.
- **Upscaling** — plates narrower than 180 px are enlarged (OpenCV bicubic on Jetson; Real-ESRGAN is supported where PyTorch allows it).

5.6 OCR — PARSeq on TensorRT

`src/ocr/parseq_tensorrt_engine.py`. **PARSeq** (Permuted Autoregressive Sequence model) is a transformer-based scene-text recogniser. The *tiny* variant, fine-tuned on plates, is compiled to TensorRT (`models/ocr/parseq_official_tiny.engine`, input 32×128).

- Uses TensorRT 10's named-tensor API and `execute_async_v3`.
- **PyCUDA** allocates the GPU input/output buffers once and reuses them (avoids per-call allocation cost).
- If the engine was built for a different GPU, it is rebuilt automatically for this one and saved.
- Alternative backends selectable in config: RapidOCR (ONNX Runtime, CPU), EasyOCR, PaddleOCR, TrOCR.

5.7 Validation, fusion and de-duplication

Step	Module	How it works
Plate validation	<code>validation/plate_validator.py</code> , <code>ocr/ocr_postprocessor.py</code>	Regular expressions for Indian plate formats (e.g. DL8CA1234) and BH series; common OCR confusions (O/0, I/1...) are corrected before re-validating.
OCR fusion	<code>validation/ocr_fusion.py</code>	Collects up to 5 readings per track and votes; at least 2 identical readings and confidence ≥ 0.55 are required. Then an OCR pass on the best upscaled crop may replace the result if more confident.
Duplicate filter	<code>database/duplicate_filter.py</code>	The same plate (or a plate within Levenshtein edit distance 2) within 30 s is not stored twice — catches a vehicle whose track split in two.

5.8 Attributes added to each detection

Attribute	Technology / method
Plate colour	HSV colour ranges on the plate crop (<code>classification/color_classifier.py</code>)
Registration category (vehicle_type)	Plate colour mapped by Indian rules: White → Private, Yellow → Commercial, Green → EV, ...
Direction (IN/OUT)	Track centroid crossing a virtual line (<code>utils/direction_detector.py</code>)
Vehicle class	YOLO class of the tracked vehicle (car/truck/bus/motorcycle)
Body colour	HSV heuristic on the vehicle crop (see section 10)

Images

Full plate crop, full vehicle crop (largest view of the track), vehicle thumbnail ≤ 320 px, plate thumbnail ≥ 176 px wide — JPEG via OpenCV

6. Multi-camera layer

Package `src/cameras/`.

Module	Responsibility
<code>registry.py</code>	Loads <code>config/camera_config.yaml</code> ; validates ids, coordinates, sources; orders cameras by order.
<code>sources.py</code>	VideoSource abstraction with UploadSource (checks the file exists and actually decodes) and RTSPSource (validates URL). Switching a camera from video to live is one config field: <code>source_type</code> .
<code>manager.py</code>	CameraManager : saves uploads, builds the queue, runs cameras one at a time on a background thread, reports progress, supports stop, deletes sessions.
<code>pipeline_models.py</code>	Loads the vehicle detector, plate detector and OCR engine once per process and shares them with every camera (loading takes 13–40 s on this hardware).
<code>cuda_context.py</code>	Pushes the CUDA primary context when a worker thread starts and pops it before the thread ends — otherwise PyCUDA aborts the whole process when processing finishes.
<code>stream_probe.py</code>	TCP reachability check for RTSP URLs (seconds instead of ~3 minutes of OpenCV retries); “Test stream” preview frame; password masking in URLs.
<code>status_store.py</code>	Writes live status as JSON with an atomic temp-file-and-rename, so the dashboard never reads a half-written file.
<code>preview_server.py</code>	MJPEG HTTP server (port 8765) for the camera wall — see section 14.

Adding and removing camera sites

Operators manage the camera list from the dashboard (**Add camera**, and **Remove camera** on each card) or over the API (POST `/trajectory-api/cameras`, DELETE `/trajectory-api/cameras/{id}`).

- **Adding** asks for the **place, latitude and longitude**. Coordinates are required, not optional: a camera without them records events that no map or heatmap can place. The new camera is complete at once — its own upload/RTSP controls, its own panel on the camera wall, its own marker on the map and heatmap, and the last place in the processing queue. It just has no source yet.
- **Renaming** changes only the label. The camera keeps its id, coordinates, queue position and source, and events already recorded keep the name they were stamped with — they record what the site was called when the vehicle passed.
- **Removing** takes the site's configuration, its uploaded video and its last preview frame, so no still image is left behind on the wall. Detections it already recorded are **kept** — they are history and carry their own camera id and coordinates, so past trajectories still plot. The last camera cannot be removed.
- **Where it is stored:** `camera_config.yaml` is hand-written and heavily commented, so it is never rewritten. The operator's camera list lives beside it in `config/cameras_runtime.yaml`, written only by code; when present it replaces the shipped camera list, while processing and `trajectory` still come from the main file. Delete it to go back to the shipped cameras.
- Neither operation is allowed while a session is running.

Why recorded files run one at a time

The Jetson has one GPU and the pipeline already uses it fully. Running cameras in parallel

would make every camera slower. Sequential processing also means, for the demo, that timestamps increase camera by camera.

Why live streams run together

A live stream carries what is happening *now*. Making camera 2 wait its turn would not delay its footage, it would discard it: every vehicle passing camera 2 during camera 1's turn would simply never be seen. So every RTSP camera starts at once, on its own thread, and keeps running until the operator presses STOP — no time limit.

They share one set of GPU models through a lock (`PipelineModels.shared()`), because a TensorRT execution context can only process one inference at a time. Decoding, tracking, OCR fusion and database writes still happen in parallel, which is where a live stream spends most of its time. Measured on this device with four cameras at once: about 7.9 fps each, roughly 31 fps in total, against about 20 fps for a single camera alone.

A queue may mix the two: the uploaded videos run through first, then the live cameras start together and stay up.

Processing sessions

Each START creates a **processing session** id (e.g. `20260913-092351-5045e9`) stored on every detection. Sessions can be viewed separately and deleted (events, images and profiles are updated).

7. Database and SQL

7.1 Database engines

Engine	When used	Connection
SQLite (default)	Single device, zero setup. File data/alpr.db.	sqlite:///data/alpr.db
PostgreSQL	Production / multiple devices writing to one server.	DB_URL=postgresql:// user:pass@host:5432/alpr_db

Which database is used, in order of priority: environment variable `DB_URL` → `ALPR_DB_PATH` → `database.path` in `config.yaml`.

7.2 SQLAlchemy ORM

SQLAlchemy 2.0 maps Python classes to tables (`src/database/models.py`). The code never writes raw SQL strings for queries; SQLAlchemy generates SQL for SQLite or PostgreSQL. Sessions are opened with a context manager that commits on success and rolls back on error (`db.get_session()`).

7.3 Table `vehicle_events` — one row per stored detection

Column	SQL type	Meaning
<code>id</code>	INTEGER PK	Auto-increment id
<code>plate_number</code>	VARCHAR	Validated plate text, e.g. DL8CA1234
<code>vehicle_type</code>	VARCHAR	Registration category from plate colour (Private, Commercial, EV...)
<code>plate_color</code>	VARCHAR	Plate background colour
<code>series_type</code>	VARCHAR	“normal” or “BH”
<code>timestamp</code>	VARCHAR	ISO-8601 UTC time
<code>direction</code>	VARCHAR	IN / OUT
<code>image_path</code>	VARCHAR	Plate crop image
<code>camera_id, camera_name</code>	VARCHAR	Which camera saw it
<code>latitude, longitude</code>	DOUBLE PRECISION	Camera location (WGS84), copied at insert time
<code>vehicle_image_path</code>	VARCHAR	Full vehicle crop
<code>trajectory_order</code>	INTEGER	Camera's queue position at capture time
<code>processing_session</code>	VARCHAR	Run id (NULL for single-gate pipeline events)
<code>video_source</code>	VARCHAR	File path or RTSP URL
<code>confidence</code>	DOUBLE PRECISION	Fused OCR confidence
<code>ocr_text</code>	VARCHAR	Raw fused OCR string before validation
<code>vehicle_class</code>	VARCHAR	YOLO class
<code>vehicle_color</code>	VARCHAR	Body colour
<code>vehicle_thumbnail_path, plate_thumbnail_path</code>	VARCHAR	Small JPEGs for cards

Indexes: `idx_plate_number`, `idx_timestamp`, `idx_camera_id`, `idx_processing_session`.

7.4 Table `vehicle_profiles` — one row per plate

Column	SQL type	Meaning
<code>plate_number</code>	<code>VARCHAR</code> <code>UNIQUE</code>	Vehicle key
<code>vehicle_class</code> , <code>vehicle_color</code> , <code>vehicle_type</code> , <code>plate_color</code>	<code>VARCHAR</code>	Majority value over all detections
<code>vehicle_image_path</code> , <code>plate_image_path</code> , <code>vehicle_thumbnail_path</code> , <code>plate_thumbnail_path</code>	<code>VARCHAR</code>	From the highest-confidence detection
<code>camera_id</code> , <code>camera_name</code> , <code>latitude</code> , <code>longitude</code> , <code>processing_session</code>	<code>VARCHAR</code> / <code>FLOAT</code>	Where it was last seen
<code>ocr_confidence</code> , <code>best_confidence</code>	<code>FLOAT</code>	Latest and best OCR confidence
<code>first_seen</code> , <code>last_seen</code>	<code>VARCHAR</code>	ISO timestamps
<code>total_detections</code> , <code>total_camera_visits</code> , <code>unique_cameras</code>	<code>INTEGER</code>	Counts
<code>trajectory_history</code>	<code>TEXT (JSON)</code>	Ordered list of camera visits
<code>attribute_votes</code>	<code>TEXT (JSON)</code>	Vote tallies behind class/colour
<code>updated_at</code>	<code>VARCHAR</code>	Last update time

7.5 Schema migration

SQLAlchemy's `create_all()` creates missing tables but never changes existing ones. `db._migrate_schema()` therefore inspects the live table and runs `ALTER TABLE vehicle_events ADD COLUMN ...` for any missing column, one statement per transaction. It is idempotent (safe to run on every start) and tolerates two processes migrating at the same time. When `vehicle_profiles` is created for the first time, profiles are built from existing history automatically.

7.6 Representative SQL (what SQLAlchemy generates)

All detections of one plate, oldest first (trajectory)

```
SELECT * FROM vehicle_events
WHERE plate_number = 'DL8CA1234' AND processing_session IS NOT NULL
ORDER BY timestamp ASC;
```

Vehicles seen at two or more cameras

```
SELECT plate_number,
       COUNT(DISTINCT camera_id) AS camera_count,
       COUNT(id)                 AS detection_count,
       MIN(timestamp) AS first_seen, MAX(timestamp) AS last_seen
FROM vehicle_events
WHERE processing_session IS NOT NULL
GROUP BY plate_number
HAVING COUNT(DISTINCT camera_id) >= 2
ORDER BY camera_count DESC, detection_count DESC;
```

Statistics per camera

```
SELECT camera_id, camera_name, COUNT(id) AS detections,  
       COUNT(DISTINCT plate_number) AS unique_plates, AVG(confidence) AS avg_confidence  
FROM vehicle_events WHERE processing_session = :session  
GROUP BY camera_id, camera_name;
```

Delete a processing session

```
DELETE FROM vehicle_events WHERE processing_session = :session;  
-- then the affected vehicle_profiles are rebuilt from the remaining events
```

Description search (e.g. “white commercial trucks”)

```
SELECT * FROM vehicle_profiles  
WHERE vehicle_color IN ('White') AND vehicle_class IN ('truck')  
      AND vehicle_type IN ('Commercial')  
ORDER BY last_seen DESC;
```

7.7 Files stored on disk

Folder	Contents
data/plate_crops/	Plate images
data/vehicle_crops/	Vehicle images
data/thumbnails/vehicles, /plates	Card-sized thumbnails
data/uploads/	Uploaded videos, one file per camera
data/live_frames/	Latest annotated frame per camera (camera wall)
data/processing_status.json	Live processing status

8. Trajectory reconstruction and GIS

Package `src/trajectory/`. Works on plain data, not on the database, so the logic is fully unit-tested.

1. **Group into visits** — consecutive detections at the same camera within 300 s become one visit; a genuine return to a camera later is a new visit.
2. **Order** — by timestamp; if any timestamp is unusable, by camera queue order (`order_by: auto`).
3. **Measure each leg** between consecutive visits:
 - **Distance** — Haversine great-circle formula on Earth radius 6371.0088 km.
 - **Bearing** — initial compass bearing (used to rotate map arrows).
 - **Duration** and **speed** = distance ÷ time (only when both exist and time > 0).
4. **Output** — ordered points, legs, summary figures, and **GeoJSON** (RFC 7946): one LineString for the path plus one Point per visit.

Coordinate order: GeoJSON uses [longitude, latitude]; Leaflet uses [latitude, longitude]. The conversion happens in one place (`trajectory/geojson.py`).

Coordinates always come from the camera location stored on each detection — no route is hard-coded.

9. Maps and the traffic heatmap

9.1 Mapping technology

Technology	Role
Leaflet.js 1.9.4	Open-source JavaScript map library, loaded from cdnjs
OpenStreetMap tiles	Street map layer
Esri World Imagery (+ reference labels)	Satellite layer
Leaflet.heat 0.2.0	Heat layer plugin
Streamlit components.html	Embeds the self-contained map page in the dashboard

The map HTML is generated in Python (`src/mapping/leaflet.py`, `src/mapping/heatmap.py`). Vehicle thumbnails are embedded as base64 data URIs because the embedded frame cannot read local files.

9.2 Trajectory map

Colour-graded polyline (green at the first camera → red at the last), direction arrows rotated to each leg's bearing, numbered markers with popups (camera, plate, time, confidence, image), street/satellite toggle, legend with path length.

9.3 Traffic heatmap

View	What each camera's value means
Where vehicles are now (default)	Number of vehicles whose <i>most recent</i> sighting was at that camera. If four vehicles passed cameras 1→2→3 and were last seen at 4, only camera 4 is red.
Total traffic per camera	Every detection recorded at that camera.

Colours run green → red with the count; grey means no vehicles. Follows the sidebar session scope.

10. Vehicle profiles and body-colour detection

10.1 Profiles

`src/database/vehicle_profiles.py`. Each detection is folded into the plate's profile inside the same database transaction, isolated in a **savepoint** so a profile error can never lose the detection. The same fold function can rebuild a profile from all events, so live and rebuilt profiles are identical.

- Class and colour are **majority votes**, ignoring “Unknown”.
- Images come from the highest-confidence detection.
- Out-of-order detections trigger a rebuild so history stays in time order.
- Deleting a session rebuilds affected profiles; a profile with no events left is deleted.

10.2 Body-colour detection

`src/classification/vehicle_color.py` — a fast HSV (Hue-Saturation-Value) colour heuristic, no extra model. Supported: White, Black, Silver, Gray, Blue, Red, Green, Yellow, Brown, Orange.

1. Skip the top 35% of the vehicle box (windscreen and roof reflect sky) and the edges.
2. Downscale to ~64 px wide and convert BGR → HSV.
3. If $\geq 25\%$ of pixels are saturated ($S \geq 55$, $V \geq 50$) the vehicle is *coloured*: vote on hue bands (brown = dark orange).
4. Otherwise it is *achromatic*: median brightness decides White (≥ 180) / Silver (≥ 130) / Gray (≥ 70) / Black.

Measured accuracy: **30 of 36** hand-labelled vehicle crops from this site's videos. Cost ≈ 0.5 ms per stored vehicle. Thresholds are configurable under `vehicle_color:` in `config.yaml`.

11. Vehicle search (natural-language parser)

`src/search/nl_query.py` turns a typed sentence into database filters. It is a **rule-based parser** using regular expressions and vocabularies, running locally — *not* a large language model (no model or API key is installed on the device). It answers instantly and offline.

Feature	Example words	Filters on
Body colour	white, grey, navy, maroon...	vehicle_color
Vehicle type	car, suv, truck, lorry, bus, bike, scooter	vehicle_class
Registration	private, commercial, taxi, electric, govt, diplomatic, army	vehicle_type
Plate colour	"yellow plate", "green number"	plate_color
Plate	DL8CA1234, "plate KA02"	plate_number
BH series	"bh series"	series_type
Direction	entering, leaving, exit	direction
Cameras visited	"3+ cameras", "more than 1 cameras", "multiple cameras"	unique_cameras
Confidence	"high confidence", "confidence above 80%"	best_confidence
Time	today, yesterday, after 9am, before 18:30, between 9 and 11am, last 2 hours, 2026-09-12	visit times
Camera	"at India Gate", CAM002	visit camera

Time and camera must be true for the **same visit**. Results show each vehicle's picture, colour, type, last matching camera and time, and a **View trajectory** button. The dashboard prints what was understood (e.g. "colour White, type car, seen after 13 Sep 09:00").

12. Blacklisted vehicles

`src/alerts/blacklist.py`, list stored in `config/blacklist.yaml` (currently **DL7CD5017**).

- **Matching** — case-insensitive, ignoring spaces and dashes; exact otherwise.
- **Live alert** — when the pipeline stores a blacklisted plate, the processing log shows “ALERT: BLACKLISTED vehicle ... detected at <camera>” and a warning is logged.
- **Dashboard banner** — red alert at the top listing each blacklisted plate, last camera, time and number of sightings (from all sessions).
- **Management** — “Blacklisted vehicles” section to add (with reason) or remove plates; view image and trajectory.
- **Badges** — red BLACKLISTED badge on search results and trajectory card.
- The file is re-read when it changes, so edits apply even during a running session.

13. REST API (FastAPI)

`src/api/server.py` and `src/api/trajectory_routes.py`. Built with **FastAPI** (on **Starlette**), request/response schemas in **Pydantic** (`src/api/schemas.py`), served by **Uvicorn**. Interactive documentation is generated automatically at `http://<device>:8000/docs` (Swagger UI) and `/redoc`.

13.1 Core endpoints

Method	Path	Purpose
POST	<code>/entry</code>	Record a vehicle event (JSON body)
GET	<code>/logs</code>	All events, newest first
GET	<code>/search?plate=</code>	Events for a plate
GET	<code>/direction?direction=IN OUT</code>	Events by direction
GET	<code>/stats</code>	Today's statistics
GET	<code>/vehicles/{plate}</code>	History of a vehicle, plus its profile
GET	<code>/live</code>	Most recent events
GET	<code>/settings</code>	Read-only configuration
GET	<code>/health</code>	Health check
POST	<code>/clear</code>	Delete all events and images
GET	<code>/snapshot</code> , <code>/stream</code>	Current frame / MJPEG live feed (single-gate)
—	<code>/media</code> , <code>/thumbnails</code>	Static image files

13.2 Multi-camera endpoints (`/trajectory-api`)

Method	Path	Purpose
GET	<code>/cameras</code>	Camera registry
POST	<code>/cameras</code>	Add a camera site (place + coordinates)
PATCH	<code>/cameras/{camera_id}</code>	Rename a camera site
DELETE	<code>/cameras/{camera_id}</code>	Remove a camera site; its events are kept
POST	<code>/cameras/upload</code>	Upload videos for all cameras at once
POST	<code>/cameras/{camera_id}/upload</code>	Upload one camera's video
POST	<code>/cameras/{camera_id}/rtsp</code>	Switch a camera to an RTSP URL
POST	<code>/processing/start</code>	Start the sequential run
POST	<code>/processing/stop</code>	Request a stop
GET	<code>/processing/status</code>	Live progress
GET	<code>/trajectory/{plate}</code>	Reconstructed path, legs and profile
GET	<code>/map/{plate}</code>	Path as GeoJSON
GET	<code>/map/{plate}/html</code>	Path as a Leaflet page
GET	<code>/vehicles</code>	Vehicle profiles, most recent first
GET	<code>/vehicles/{plate}</code>	One vehicle profile
GET	<code>/statistics</code>	Per-camera and total statistics

GET	/plates	Plates seen at N+ cameras
GET	/sessions	Processing sessions
DELETE	/sessions/{session_id}	Delete a session and its events

13.3 Example

```
curl http://localhost:8000/trajectory-api/trajectory/KA02MH7256
{
  "plate_number": "KA02MH7256",
  "path_labels": ["India Gate", "Connaught Place"],
  "cameras_visited": 2,
  "total_distance_km": 2.4163,
  "legs": [{
    "from_camera_name": "India Gate",
    "to_camera_name": "Connaught Place",
    "distance_km": 2.4163,
    "duration_seconds": 37.0,
    "bearing_degrees": 328.9,
    ...
  }],
  "profile": {
    "vehicle_class": "car",
    "vehicle_color": "Blue",
    "total_camera_visits": 2,
    ...
  }
}
```

14. Dashboards and live video

14.1 Multi-camera dashboard (main) — Streamlit

`src/dashboard/trajectory_app.py`, port 8501. Streamlit re-runs the Python script on every interaction and renders widgets as a web page; long-lived objects (camera manager, models, preview server) are kept in module-level singletons so they survive reruns.

The console is split into three pages, each with one job. Home is the page an operator leaves open all day, so it carries nothing they only touch when starting a run; Operations carries exactly those controls.

Page	Sections, in order
Home watching	Blacklisted vehicles — alert banner, watch list, add/remove plates, sightings Camera feeds — live video wall, four feeds per row Search vehicle by plate number — with a “seen at 2+ cameras” picker Search vehicles by description — image results; click one for its route Traffic heatmap — “Where vehicles are now” or “Total traffic per camera”
Operations running	The three sections stacked full width, in the order the work happens: Cameras — four camera cards per row (a fifth starts the next row), each with Upload video <i>or</i> RTSP stream, Set/Test stream, Clear, Rename, Remove; then Add camera and START/STOP PROCESSING Live Processing Status — current camera, FPS, progress bars, time remaining, current plate and vehicle, processing log Statistics — vehicles detected, unique plates, processing time, average OCR and detection time, detections per camera
Trajectory one vehicle	Vehicle and plate images, type, colour, metrics, timeline, route map, detection history, GeoJSON download
Sidebar (all pages)	Auto-refresh, session scope, delete session, camera network, reload config

Either search on Home opens the vehicle on the Trajectory page.

14.2 Other dashboards

- **Original Streamlit dashboard** — `src/dashboard/app.py`: live feed cards, search, analytics (Plotly charts), vehicle history, CSV export.
- **Web dashboard** — `src/dashboard_web/` (HTML, CSS, JavaScript), served by FastAPI at `/`.

14.3 Smooth live video — MJPEG preview server

- The pipeline's **LiveFramePublisher** writes each camera's annotated frame to `data/live_frames/<camera>.jpg` at up to 15 fps, downscaled to 960 px, encoded on a **background thread** so ALPR is not slowed.
- The **preview server** (Python `http.server`, port 8765) streams those files as **MJPEG** (`multipart/x-mixed-replace`), which browsers play natively in an `<img>` tag.
- The player HTML never changes between Streamlit reruns, so the video keeps playing continuously.
- Measured: 14.1 fps (recorded video) and 13.6 fps (RTSP) delivered to the browser, versus 0.5 fps before.

15. Configuration files

File	Purpose	Key settings
config/config.yaml	Pipeline, database, API, dashboard tuning	video (frame_skip, retries), detection (model paths, thresholds, FP16), tracking, preprocessing, enhancement, ocr (backend, PARSeq engine), fusion, deduplication, vehicle_color, color_classifier, direction, database (path, image and thumbnail folders), api (ports, live frames, preview port), logging, training
config/camera_config.yaml	Camera network	Per camera: camera_id, camera_name, latitude, longitude, video_path, rtsp_url, source_type, enabled, order; processing (upload dir, status file, continue_on_error); trajectory (order_by, collapse, revisit gap)
config/cameras_runtime.yaml	Cameras added or removed from the dashboard	Written by the app; replaces the shipped camera list. Delete it to restore the four shipped cameras.
config/blacklist.yaml	Watch list	plate, reason, added
.env	Local overrides (not in git)	ALPR_VIDEO_SOURCE, ALPR_DB_PATH, DB_URL

Camera sites

ID	Name	Latitude	Longitude
CAM001	India Gate	28.6129	77.2295
CAM002	Connaught Place	28.6315	77.2167
CAM003	Karol Bagh	28.6519	77.1909
CAM004	Kashmere Gate	28.6675	77.2273

16. Project structure

```
alpr-university-gate/
├── config/                config.yaml · camera_config.yaml · blacklist.yaml
├── scripts/run_pipeline.py  ALPR pipeline (CLI entry: python main.py)
├── src/
│   ├── capture/          OpenCV frame capture (files, RTSP, webcam)
│   ├── detection/        YOLOv8 vehicle and plate detectors
│   ├── tracking/         ByteTrack vehicle tracker
│   ├── preprocessing/    plate image preprocessing
│   ├── enhancement/      super-resolution / upscaling
│   ├── ocr/              PARSeq TensorRT, RapidOCR, other OCR backends
│   ├── validation/       plate validation, OCR fusion
│   ├── classification/   plate colour, registration category, body colour
│   ├── database/         SQLAlchemy models, queries, migration, profiles
│   ├── cameras/          registry, sources, manager, preview server, CUDA context
│   ├── trajectory/       trajectory engine, GeoJSON
│   ├── mapping/          Leaflet trajectory map, heatmap
│   ├── search/           natural-language vehicle query parser
│   ├── alerts/           blacklist
│   ├── api/              FastAPI server, routes, schemas
│   ├── dashboard/        Streamlit dashboards
│   ├── dashboard_web/    HTML/JS web dashboard
│   └── utils/            config, logging, live frames, Jetson paths
├── models/              TensorRT engines and weights
├── data/                SQLite database, images, uploads, live frames
├── tests/               unit and integration tests
├── training/            model training scripts
└── docs/                documentation
```

Size: about 17,300 lines of Python in `src/` and `scripts/`.

17. Running the system

17.1 Processes and ports

Process	Port	Start command
Multi-camera dashboard (includes processing)	8501	streamlit run src/dashboard/trajectory_app.py
Camera wall video streams (started by dashboard)	8765	automatic
REST API + web dashboard	8000	uvicorn src.api.server:app --host 0.0.0.0 --port 8000
Single-gate pipeline (optional)	—	python main.py or python scripts/run_pipeline.py --source ALPR.mp4
Original dashboard (optional)	8501	streamlit run src/dashboard/app.py

Activate the environment first: `source /home/ansh/alpr/alpr_env/bin/activate`, then run commands from the project folder.

17.2 Typical workflow

1. Open `http://<jetson-ip>:8501`.
2. For each camera choose **Upload video** or **RTSP stream** (use **Test stream** to check a camera).
3. Press **START PROCESSING** and watch the live status and camera wall.
4. Search a plate or describe a vehicle; click **View trajectory**.
5. Review the heatmap, statistics, blacklist alerts; delete old sessions from the sidebar.

17.3 Moving a camera to live RTSP

```
- camera_id: "CAM001"
  source_type: "rtsp"
  rtsp_url: "rtsp://user:password@192.168.1.100:554/11"
```

Nothing else changes. Every live camera runs together with the others, continuously, until the session is stopped.

18. Measured performance (Jetson Orin)

Measurement	Result
Model load (once per process)	13–40 s (plus ~3 min once if the OCR engine is rebuilt for a new GPU)
Pipeline speed, ALPR.mp4	≈ 19–22 fps
Pipeline speed, mycarplate.mp4 (heavier)	≈ 2 fps
Pipeline speed, live RTSP camera	≈ 20–24 fps
Four cameras running together (shared models)	≈ 7.9 fps each, ≈ 31 fps in total
Camera wall video in browser	13.6–14.1 fps
Body colour detection	≈ 0.5 ms per stored vehicle
Thumbnails (both)	≈ 1.3 ms per stored vehicle
Profile update	≈ +14 ms on each database insert
Unreachable RTSP camera detected	≤ 3 s (was ≈ 3 min)

19. Testing

Tool	Used for
pytest 8.2.0	Test runner — 665 automated tests (tests/unit, tests/integration)
Hypothesis 6.100.0	Property-based tests (random inputs) for validation logic
FastAPI TestClient	Calls every API endpoint against temporary databases
Streamlit AppTest	Renders the dashboard and clicks through it (uploads, RTSP, search, delete, blacklist)
Real-footage checks	End-to-end runs on ALPR.mp4, mycarplate.mp4 and the live RTSP camera; 36 hand-labelled crops for colour accuracy

Run: `python -m pytest tests/ -q`.

20. Known limitations

- **Vehicle search is rule-based**, not a large language model; it understands the vocabulary in section 11, not arbitrary sentences.
- **Body colour is a heuristic** (30/36 on real crops); watermarks, reflections and partly hidden vehicles can mislead it.
- **Blacklist matching is exact**; an OCR misread of a blacklisted plate will not trigger an alert.
- **Uploaded videos are processed one at a time**, so a long queue of recordings takes the sum of their runtimes. Live cameras do run together and continuously; they share one GPU, so each stream runs slower than a single camera would.
- **Demo speeds look unrealistic** (hundreds of km/h) because one video is replayed at sites kilometres apart; real cameras give real travel times.
- **Clock sync**: trajectories order by timestamps, so live cameras/devices need NTP time synchronisation.
- The camera wall uses a second port (8765); it must be reachable from viewing computers.
- At process exit PyCUDA may print an “Exception ignored” message; it does not affect data or the running dashboard.

21. Glossary

Term	Meaning
ALPR / ANPR	Automatic Licence (Number) Plate Recognition
YOLO	“You Only Look Once” — real-time object detection neural network
COCO	Large public image dataset with 80 object classes, used to pretrain YOLO
TensorRT engine	A model compiled and optimised for one specific NVIDIA GPU
FP16	16-bit floating point; halves memory and speeds up GPU inference
CUDA	NVIDIA's platform for running computations on the GPU
ByteTrack	Multi-object tracking algorithm that links detections across frames
OCR	Optical Character Recognition — reading text from images
PARSeq	Transformer-based scene-text recognition model
CLAHE	Contrast-Limited Adaptive Histogram Equalisation
Levenshtein distance	Number of single-character edits between two strings
HSV	Hue-Saturation-Value colour space, convenient for colour rules
RTSP	Real Time Streaming Protocol, used by IP cameras
MJPEG	Motion JPEG — video sent as a stream of JPEG images over HTTP
ORM	Object-Relational Mapper (SQLAlchemy) — Python objects ↔ database rows
REST API	HTTP interface returning JSON
ASGI	Asynchronous Server Gateway Interface — how Uvicorn runs FastAPI
GeoJSON	Standard JSON format for geographic features (RFC 7946)
WGS84	World coordinate system used by GPS (latitude/longitude)
Haversine	Formula for distance between two latitude/longitude points on a sphere
Processing session	One START-to-finish run over the camera queue
Vehicle profile	One summary record per plate across all detections
Savepoint	A nested transaction that can be rolled back without undoing the outer one